

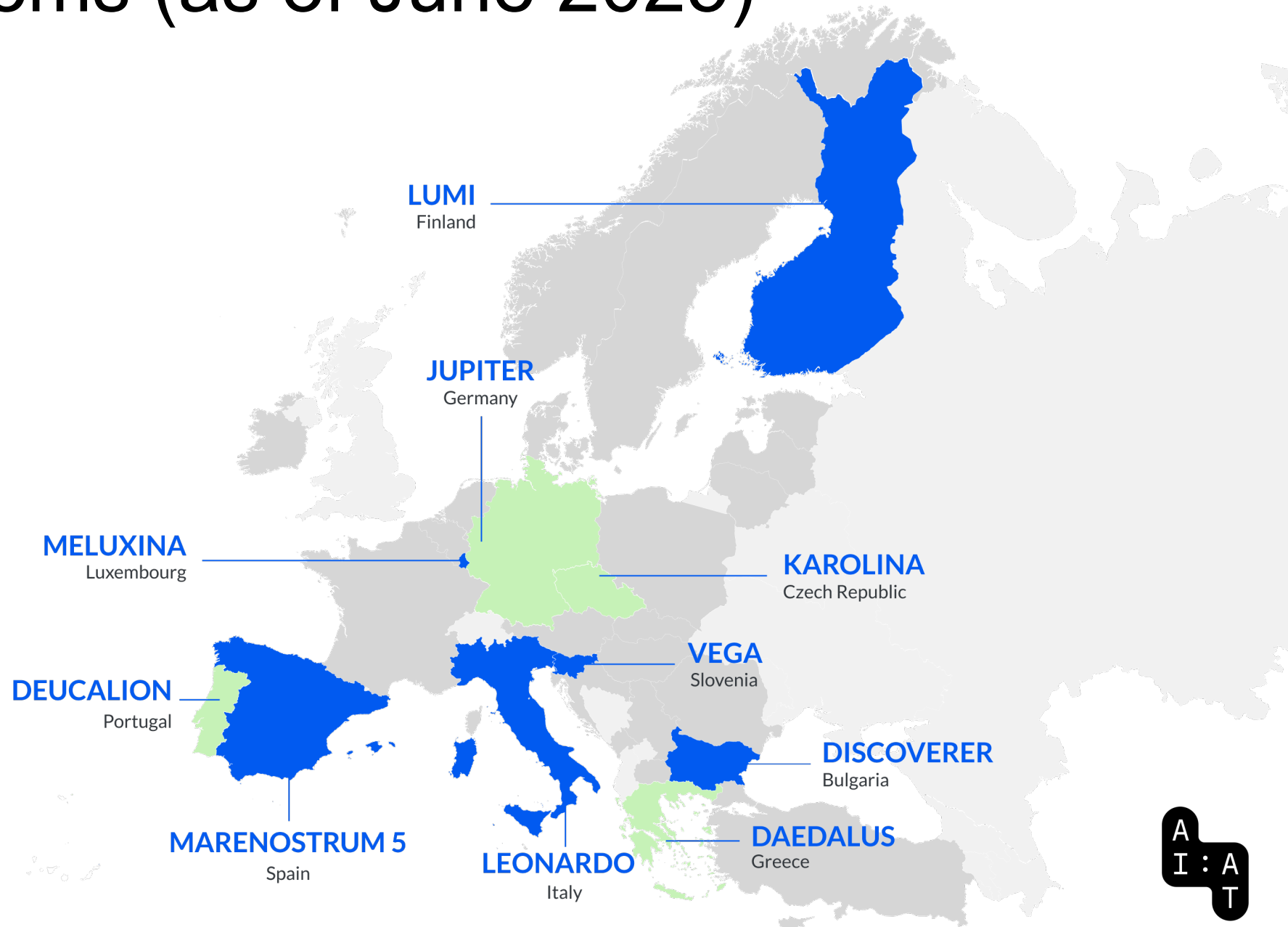
Supercomputer Access at the Hackathon

EuroHPC Systems (as of June 2025)

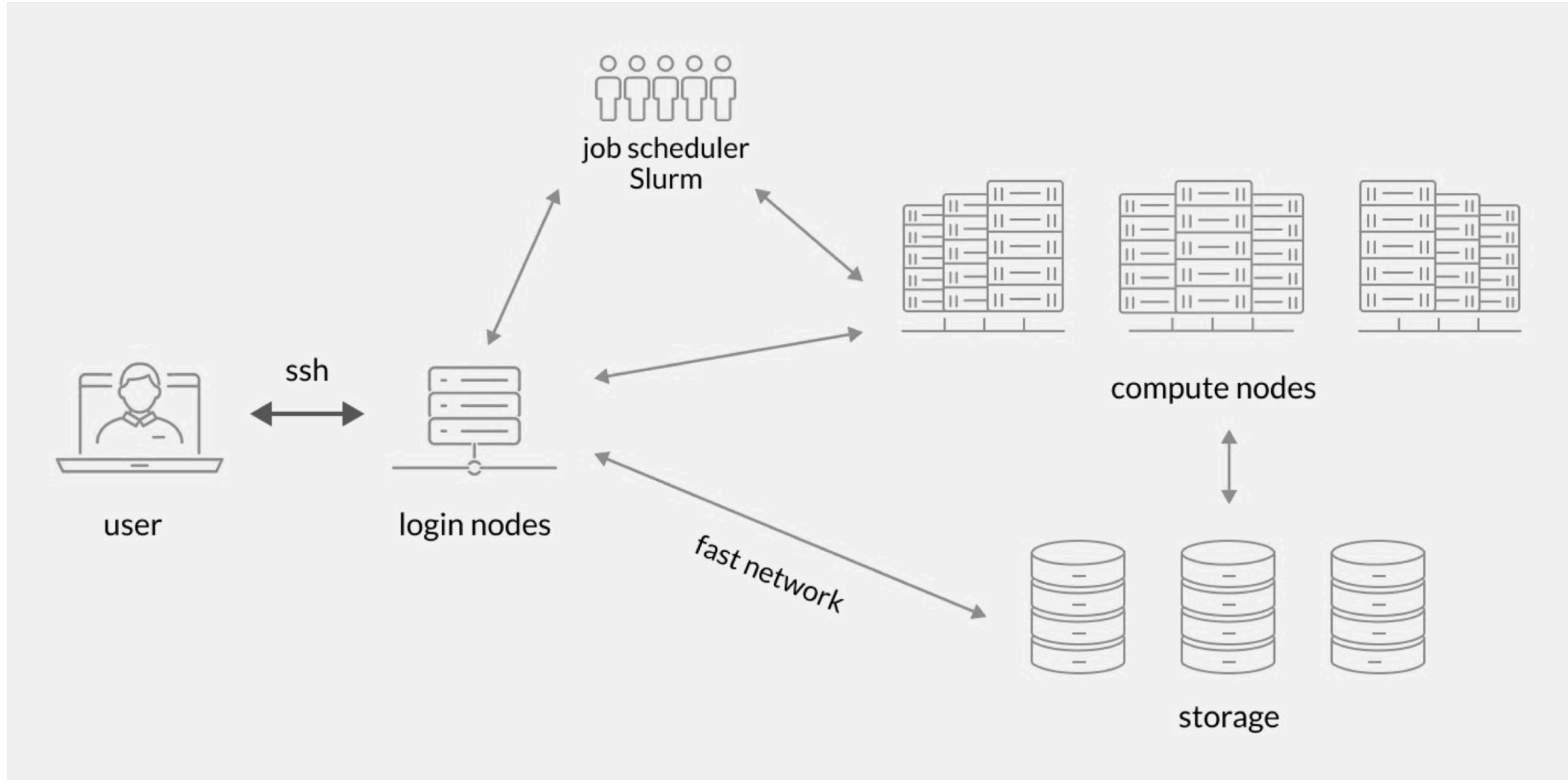
In Top 500 list

<https://www.top500.org/>

- #4 JUPITER
Nvidia GH200
- #9 LUMI
AMD MI250X
- #10 LEONARDO
Nvidia A100



Typical Setup of a Supercomputer



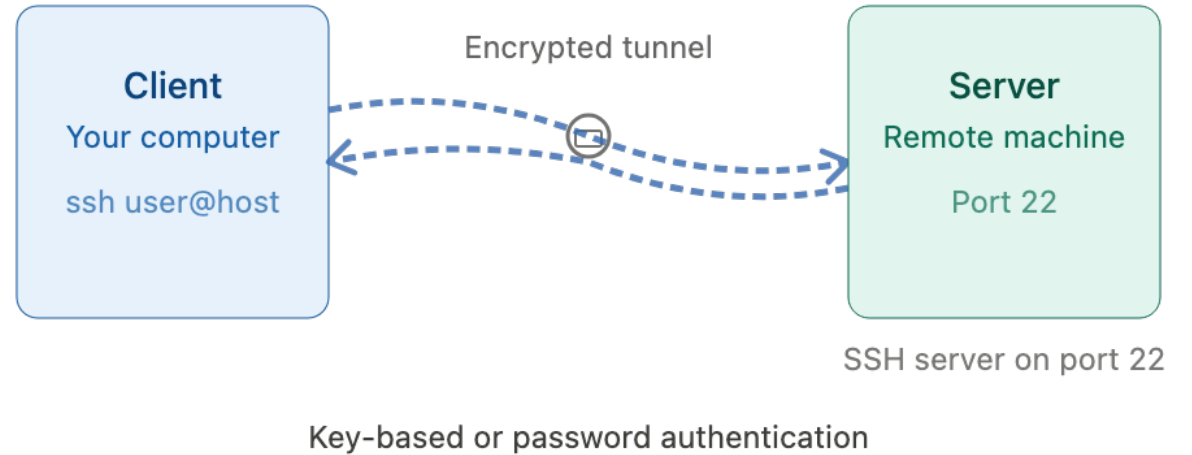
SSH: Secure Shell

What is it?

- Remote access protocol
- End-to-end encrypted

What can you do with it?

- Run commands / get a shell
- File transfer (SCP/SFTP)
- Tunnels / port forwarding



SSH Access to Leonardo

Use any of the following login nodes:

```
ssh your_username@login01-ext.leonardo.cineca.it
```

```
ssh your_username@login02-ext.leonardo.cineca.it
```

```
ssh your_username@login05-ext.leonardo.cineca.it
```

```
ssh your_username@login07-ext.leonardo.cineca.it
```

(For the hackathon, two factor authentication is not used.)

Pixi Package Manager

- Fast, modern, and reproducible package manager
- Can install packages from:
 - PyPI (like pip)
 - conda-forge (like conda)
- <https://pixi.sh/>

Quick intro:

```
curl -fsSL https://pixi.sh/install.sh | bash
pixi init hello-world
cd hello-world
pixi add python # install from conda-forge
pixi add --pypi openai # install from PyPI
pixi run python -c 'print("Hello world!")'
```



Containers on HPC: Singularity / Apptainer

Most HPC systems don't run Docker, but Singularity or Apptainer (which are highly related and behave nearly identically)

<https://docs.sylabs.io/guides/latest/user-guide/>



Convert Docker containers on Leonardo:

```
srun --partition=lrd_all_serial --time 04:00:00 --gres=tmpfs:100G --  
mem=16G --pty singularity pull vllm-openai-v0.21.0-cu129.sif  
docker://docker.io/vllm/vllm-openai:0.21.0-cu129
```

Run something inside a Singularity container:

```
singularity exec --nv --bind $SCRATCH:/scratch container.sif python3
```

Slurm: Job Scheduler

Leonardo job script (1 GPU)

```
#!/bin/bash
#SBATCH --partition=boost_usr_prod
#SBATCH --reservation=s_tra_ncc # Reservation for the hackathon
#SBATCH --nodes=1 # Number of nodes
#SBATCH --ntasks-per-node=1 # Number of `srun` tasks executed per node
#SBATCH --gpus-per-task=1 # Number of GPUS (up to 4 on Leonardo)
#SBATCH --mem=120GB # Fair share on Leonardo: 120GB * gpus-per-task
#SBATCH --cpus-per-task=8 # Fair share on Leonardo: 8 * gpus-per-task
#SBATCH --time=0:30:00 # Time limit in HH:MM:SS, up to 24:00:00

# whatever command should be executed on the compute node.
```


Slurm: Job Scheduler

Leonardo job script (1 GPU)

```
#!/bin/bash
#SBATCH --partition=boost_usr_prod
#SBATCH --reservation=s_tra_ncc # Reservation for the hackathon
#SBATCH --nodes=1 # Number of nodes
#SBATCH --ntasks-per-node=1 # Number of `srun` tasks executed per node
#SBATCH --gpus-per-task=1 # Number of GPUs (up to 4 on Leonardo)
#SBATCH --mem=120GB # Fair share on Leonardo: 120GB * gpus-per-task
#SBATCH --cpus-per-task=8 # Fair share on Leonardo: 8 * gpus-per-task
#SBATCH --time=0:30:00 # Time limit in HH:MM:SS, up to 24:00:00

# Construct run command to execute inside pixi environment:
export RUN_COMMAND="/path/to/pixi run --as-is [--manifest-path pixi_project_path]"

$RUN_COMMAND python3 script.py
```

Slurm: Job Scheduler

Leonardo job script (1 GPU)

```
#!/bin/bash
#SBATCH --partition=boost_usr_prod
#SBATCH --reservation=s_tra_ncc # Reservation for the hackathon
#SBATCH --nodes=1 # Number of nodes
#SBATCH --ntasks-per-node=1 # Number of `srun` tasks executed per node
#SBATCH --gpus-per-task=1 # Number of GPUs (up to 4 on Leonardo)
#SBATCH --mem=120GB # Fair share on Leonardo: 120GB * gpus-per-task
#SBATCH --cpus-per-task=8 # Fair share on Leonardo: 8 * gpus-per-task
#SBATCH --time=0:30:00 # Time limit in HH:MM:SS, up to 24:00:00

# Construct command to run container:
export CONTAINER="singularity exec --nv container.sif"

$CONTAINER python3 script.py
```

Slurm: Job Scheduler

Leonardo job script (2 GPUs)

```
#!/bin/bash
#SBATCH --partition=boost_usr_prod
#SBATCH --reservation=s_tra_ncc # Reservation for the hackathon
#SBATCH --nodes=1 # Number of nodes
#SBATCH --ntasks-per-node=1 # Number of `srun` tasks executed per node
#SBATCH --gpus-per-task=2 # Number of GPUs (up to 4 on Leonardo)
#SBATCH --mem=240GB # Fair share on Leonardo: 120GB * gpus-per-task
#SBATCH --cpus-per-task=16 # Fair share on Leonardo: 8 * gpus-per-task
#SBATCH --time=0:30:00 # Time limit in HH:MM:SS, up to 24:00:00

# Construct command to run container:
export CONTAINER="singularity exec --nv container.sif"

$CONTAINER python3 script.py
```

Slurm: Job Scheduler

Leonardo job script (4 GPUs)

```
#!/bin/bash
#SBATCH --partition=boost_usr_prod
#SBATCH --reservation=s_tra_ncc # Reservation for the hackathon
#SBATCH --nodes=1 # Number of nodes
#SBATCH --ntasks-per-node=1 # Number of `srun` tasks executed per node
#SBATCH --gpus-per-task=4 # Number of GPUs (up to 4 on Leonardo)
#SBATCH --mem=480GB # Fair share on Leonardo: 120GB * gpus-per-task
#SBATCH --cpus-per-task=32 # Fair share on Leonardo: 8 * gpus-per-task
#SBATCH --time=0:30:00 # Time limit in HH:MM:SS, up to 24:00:00

# Construct command to run container:
export CONTAINER="singularity exec --nv container.sif"

$CONTAINER python3 script.py
```

Slurm: Job Scheduler

Leonardo job script (2 nodes with 4 GPUs each)

```
#!/bin/bash
#SBATCH --partition=boost_usr_prod
#SBATCH --reservation=s_tra_ncc # Reservation for the hackathon only enough for
1 node per team
#SBATCH --nodes=2                # Number of nodes
#SBATCH --ntasks-per-node=1      # Number of `srun` tasks executed per node
#SBATCH --gpus-per-task=4        # Number of GPUs (up to 4 on Leonardo)
#SBATCH --mem=480GB              # Fair share on Leonardo: 120GB * gpus-per-task
#SBATCH --cpus-per-task=32       # Fair share on Leonardo: 8 * gpus-per-task
#SBATCH --time=0:30:00           # Time limit in HH:MM:SS, up to 24:00:00

# Construct command to run container:
export CONTAINER="singularity exec --nv container.sif"

srun $CONTAINER python3 script.py
```

Slurm: Job Scheduler

Useful Slurm commands

Submit a job:

\$ sbatch job.sh

Check submitted jobs:

\$ squeue --me

Look at the output from a job:

\$ cat slurm-<job_id>.out

or follow the output as the job runs:

\$ tail -c +0 -f slurm-<job_id>.out

Cancel job:

\$ scancel <job_id>

Get a shell at a node while a job is running:

\$ srun --overlap --pty --jobid=<job_id> bash

Leonardo

File storage:

Shared storage between all login and compute nodes.

- `$HOME`: 50 GB limit
- `$SCRATCH`: Higher limit. Use this for larger files during the hackathon. (Files are deleted after 40 days.)
- `$PUBLIC`: 50 GB limit. Can be used to share files between Leonardo users.
- `$FAST` and `$WORK`: Do not use during the hackathon.

Login node CPU time limit:

Process on login nodes have a 10 minute CPU time limit.

Use this for longer processes:

```
srun --partition=lrd_all_serial --time 04:00:00 --gres=tmpfs:100G --mem=16G  
--pty bash
```

Leonardo internet access

- Use login nodes for large file downloads.
- Compute nodes do not have internet access.
- As a workaround, set the following environment variables in your Slurm script:

```
export HTTP_PROXY=http://proxyuser:5dd1d2bd00@10.99.0.1:38425
export HTTPS_PROXY=http://proxyuser:5dd1d2bd00@10.99.0.1:38425
export http_proxy=http://proxyuser:5dd1d2bd00@10.99.0.1:38425
export https_proxy=http://proxyuser:5dd1d2bd00@10.99.0.1:38425
```

The proxy will restart every once in a while (due to the 10 min CPU time limit).
TCP connections will drop shortly.

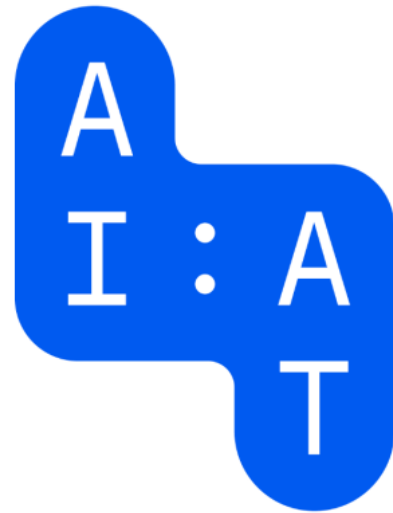
Please only use the proxy for low-bandwidth traffic. Always download large files from the login nodes.

Additional Material

Our HPC Onboarding Kit contains additional information:

- Chapter 5: First steps on LEONARDO
- Chapter 6: Software

<https://ai-at.eu/hpc-onboarding/>



Supercomputer Onboarding Kit

An AI:AT Tutorial



Funded by



EuroHPC
Joint Undertaking

 Federal Ministry
Innovation, Mobility
and Infrastructure
Republic of Austria



**Funded by
the European Union**



AI Factory Austria AI:AT has received funding from the European High-Performance Computing Joint Undertaking (JU) under grant agreement No 101253078. The JU receives support from the Horizon Europe Programm of the European Union and Austria (BMIMI / FFG).